Zadanie 3 23-26 marca 2026 r.

kurs języka C++

zmienna z historią zmian wartości

Instytut Informatyki
Uniwersytetu Wrocławskiego Paweł Rzechonek

Prolog

Cofanie i przywracanie zmian w edytorze tekstu to kluczowe funkcje, które umożliwiają efektywną pracę z dokumentami. Kombinacja klawiszy *Ctrl+Z* to jedno z najczęściej używanych narzędzi do cofania ostatnich działań w wielu programach. Dzięki niej można szybko odwołać ostatnią akcję, co jest niezwykle przydatne podczas edycji tekstów czy pracy nad grafiką. Z kolei kombinacja *Ctrl+Y* to skrót klawiszowy, który umożliwia przywracanie cofniętych czynności – działa odwrotnie do *Ctrl+Z*, pozwalając użytkownikowi odzyskać wcześniej wycofane zmiany, co jest nieocenione, gdy przypadkowo cofniemy zbyt wiele operacji i chcemy je odtworzyć.

Jednak skuteczność tych skrótów zależy od konkretnego oprogramowania oraz jego ustawień dotyczących historii zmian. Mimo tych ograniczeń Ctrl+Z i *Ctrl+Y* pozostają niezbędnymi narzędziami dla każdego użytkownika komputera, ułatwiającymi zarządzanie procesem twórczym i redukującym ryzyko utraty danych przez przypadkowe działania.

Zadanie

Zdefiniuj klasę `zmienna`, która będzie opakowaniem dla zmiennej typu `double` o określonej nazwie typu `const string`. Opakowanie to ma dodatkowo przechowywać historię zmian wartości przechowywanych w zmiennej oraz umożliwiać przywracanie poprzednich wartości pobranych z historii.

Maksymalna długość historii ma zostać określona za pomocą pewnej stałej statycznej w klasie `zmienna` (do celów testowych ustaw jej wartość na niskim poziomie, na przykład 3). Samą historię przechowywanych w zmiennej wartości zaimplementuj na tablicy utworzonej dynamicznie na stercie operatorem `new[]` (tablica ta ma być utworzona w konstruktorze, ale pamiętaj by w destruktorze zwolnić przydzieloną pamięć operatorem `delete[]`). Jeśli historia przekracza zdefiniowaną długość, to początkowe wpisy mają być bezpowrotnie tracone.

Przywracanie wartości zmiennej z historii ma być oparte na strategii LIFO (stos) – bieżąca wartość jest zastępowana wartością odczytaną z bufora historii (zawsze odczytujemy najpóźniej wpisaną wartość). Zadbaj o to, aby historia zmiennej nigdy nie była pusta (w buforze na dane historyczne zawsze musi znajdować się co najmniej jedna wartość)

– przywracanie wartości z historii z pojedynczą wartością w buforze (jest to więc bieżąca wartość zmiennej) nie powoduje żadnej zmiany w obiekcie typu `zmienna`.

Postaraj się zaimplementować operacje na historii optymalnie – bez względu na długość bufora operacje wstawienia nowej wartości do zmiennej i przywrócenia starej wartości z historii powinny wykonywać się w stałym czasie, niezależnym od długości tablicy przechowującej dane historyczne. Programując bufor wykorzystaj zawinięcie w tablicy (następny po ostatnim elemencie jest pierwszy element w tablicy), co przyspieszy działanie tej struktury.

Implementacja zmiennej z danymi historycznymi, czyli klasa `zmienna`, ma posiadać cztery konstruktory: konstruktor z zadaną nazwą i wartością, konstruktor bezparametrowy i jednocześnie delegatowy (przekazuje do poprzednio wspomnianego konstruktora wartość 0 i nazwę „x"), konstruktor kopiujący (pobierana jest tylko nazwa i bieżąca wartość z wzorcowej zmiennej bez historii) i konstruktor przenoszący. Aby uzupełnić semantykę kopiowania i przenoszenia zdefiniuj odpowiednie operatory przypisania (przypisanie kopiujące i przenoszące), przy czym każde przypisanie powoduje automatyczne dodanie kolejnego wpisu do historii zmian (nie wolno zmienić nazwy zmiennej).

Do zmiennej dopisz jeszcze dwie funkcje składowe – jedna zmieniająca znak zmiennej ($x \to -x$) i druga odwracająca jej wartość ($x \to 1/x$).

Na koniec napisz program, który rzetelnie przetestuje działanie obiektów typu `zmienna`.

Ważne elementy programu

- Podział programu na plik nagłówkowy (np. `zmienna.hpp`) z definicją klasy reprezentującej zmienną z danymi historycznymi, plik źródłowy (np. `zmienna.cpp`) z definicjami funkcji składowych dla zmiennej oraz plik źródłowy z funkcją `main()` (np. `main.cpp`).
- Obiekt zmiennej ma być inicjalizowany na kilka różnych sposobów: konkretną wartością, domyślnie (konstruktor delegatowy) za pomocą 0.0, przez skopiowanie z innej zmiennej (konstruktor kopiujący) oraz za pomocą przeniesienia zawartości ze zmiennej tymczasowej (konstruktor przenoszący).
- Utworzenie tablicy pomocniczej dla wartości historycznych na stercie w konstruktorze a potem usunięcie tej tablicy w destruktorze w klasie `zmienna`.
- Obiekt zmiennej z historią ma być kopiowalny (przypisanie kopiujące i przenoszące).
- Zgłaszanie wyjątków w konstruktorach i funkcjach składowych.
- Program powinien zostać napisany w jakimś przyjaznym IDE (na przykład *CLion* firmy *JetBrains*).
- W funkcji `main()` należy przetestować działanie zmiennej z danymi historycznymi.